



Spring

TP

En se basant sur le TP précédent:

- Créer les Rest Controller.
- Installer l'outil Postman
- Tester en utilisant Postman les différentes méthodes
- Le compte rendu doit contenir le code source et les résultats des tests avec Postman

1. Créer et configurer un Projet **Spring Boot**:

- Maven /Jar / gérer par: tn.enicarthage / livrable nommé : tp7-groupeX-numN / nom du projet : tp7-spring-boot-data-web
- Version du Java 1.8
- Dépendances: MySQL Driver / Spring Data JPA / Spring Web / Spring Boot DevTools / Lombok [facultatif] /

2. Analyser le diagramme de classes suivant et créer les :

- entités
- associations
- repositories
- services (à développer les méthodes décrites dans la page suivante)

3. Règles à respecter :

- Le démarrage de l'application devra invoquer la création du schéma de la base de données.
- Le lancement des Web Services devra alimenter la base de données
- Une bonne qualité du code est nécessaire.

4. Configurer Log4j et activer la journalisation.

5. Utiliser la classe de teste pour tester vos services.

6. Un exemple de la classe « application.properties » à utiliser

❑ ServiceEntreprise

- méthodes CRUD
- `public int ajouterEntreprise(Entreprise entreprise)`
- `public void deleteEntrepriseById(int entrepriseId)`
- `public Entreprise getEntrepriseById(int entrepriseId)`
- `public int ajouterDepartement(Departement dep)`
- `public void affecterDepartementAEntreprise(int depId, int entrepriseId)`
- `public List<String> getAllDepartementsNamesByEntreprise(int entrepriseId)`
- `public void deleteDepartementById(int depId)`

❑ ServiceEmployé

- méthodes CRUD
- `public Employe authenticate(String login, String password) ;`
- `public void mettreAJourEmailByEmployeId(String email, int employeId);`
- `public void affecterEmployeADepartement(int employeId, int depId);`
- `public void desaffecterEmployeDuDepartement(int employeId, int depId);`
- `public String getEmployePrenomById(int employeId);`
- `public void deleteEmployeById(int employeId);`
- `public int getNombreEmployeJPQL();`
- `public List<String> getAllEmployeNamesJPQL();`
- `public List<Employe> getAllEmployeByEntreprise(Entreprise entreprise);`
- `public void mettreAJourEmailByEmployeIdJPQL(String email, int employeId);`
- `public float getSalaireByEmployeIdJPQL(int employeId);`
- `public Double getSalaireMoyenByDepartementId(int departementId);`
- `public List<Employe> getAllEmployes();`
- `int addOrUpdateEmploye(Employe employe);`

❑ ServiceDépartement

- méthodes CRUD

application.properties

- #Web configuration
- spring.mvc.view.prefix=/views/
- spring.mvc.view.suffix=.jsp
- #Server configuration
- server.servlet.context-path=/SpringMVC
- spring.mvc.servlet.path=/servlet
-
- server.port=8081
- ### DATABASE ###
- #spring.datasource.driver-class-name=com.mysql.jdbc.Driver
- spring.datasource.url=jdbc:mysql://localhost:3306/BD
- spring.datasource.username=user
- spring.datasource.password=pwd
- ### JPA / HIBERNATE ###
- spring.jpa.show-sql=true
- spring.jpa.hibernate.ddl-auto=update
- spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL5InnoDBDialect
- #logging configuration
- logging.level.com.zaxxer.hikari=warn
- logging.level.org.springframework=warn
- logging.file.name=C:\\logs\\spring.log
- logging.level.root=info
- logging.pattern.console=%d{yyyy-MM-dd HH:mm:ss} - %-5level - %logger{60} - %msg%n
- logging.pattern.file=%d{yyyy-MM-dd HH:mm:ss} - %-5level - %logger{60} - %msg%n

- Diagramme de classes

